VINUNIVERSITY

Lab 03: Objects, constructors, constructor calls, static members in Java

March 16, 2026

College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring 2026

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should upload your .java file(s) to Codeforces before the end of the laboratory session unless the instructor gave a specified deadline. No Canvas or Codeforces submission will get 0 point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:

  1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
  2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
  3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

- **Navigate to the Lab:** Open the specific link provided for this week's lab: `https://vinuni26oop.contest.codeforces.com/`.

Problem 1: Vehicle Distance Tracker

Design a Java program that models different types of vehicles and computes the distance they travel.

You must create a file named `VehicleTracker.java`. The program should demonstrate the use of an abstract class, inheritance, and a static counter.

Class Requirements

Abstract Class: `Vehicle`

- Instance variables: `speed` (km/h), `time` (hours)
- Static variable to count how many vehicles are created
- Constructor to initialize speed and time
- Method `getDistance()` that returns the distance travelled
- Abstract method `printInfo()`

Subclass: `Car`

- Inherits from `Vehicle`
- Constructor receives speed and time
- Implements `printInfo()`

Subclass: `Bicycle`

- Inherits from `Vehicle`
- Constructor receives speed and time
- Implements `printInfo()`

Input

The input consists of several lines:

- First line: number of cars
- Next lines: speed and time for each car
- Next line: number of bicycles
- Next lines: speed and time for each bicycle

Output

For each vehicle created, print two lines describing the vehicle type and the distance it has travelled.

- For each **car**, print:

```
Car travelling at <speed> km/h
Distance: <distance>
```

- For each **bicycle**, print:

```
Bicycle travelling at <speed> km/h
Distance: <distance>
```

- The value <distance> is computed as: distance = speed × time

After printing the information of all vehicles, print the total number of vehicles created in the following format: <N> vehicles created

Example

Examples

Test Case 1

```
Input:
2
60 2
80 1
1
20 3

Output:
Car travelling at 60.0 km/h
Distance: 120.0
Car travelling at 80.0 km/h
Distance: 80.0
Bicycle travelling at 20.0 km/h
Distance: 60.0
3 vehicles created
```

Test Case 2

```
Input:
1
50 4
2
15 2
18 3

Output:
Car travelling at 50.0 km/h
Distance: 200.0
Bicycle travelling at 15.0 km/h
Distance: 30.0
Bicycle travelling at 18.0 km/h
Distance: 54.0
3 vehicles created
```

Problem 2: Library Membership System

A library keeps track of its members and their membership details.

Create a Java program named `LibraryMemberApp.java` that demonstrates the use of abstract classes, constructor, inheritance, and interfaces.

Class Requirements

Abstract Class: `Library`

- Static variable: `libraryName = "Central Library"`
- Instance variables: `city`, `section`
- Constructor initializes the library location
- Method `printLibraryInfo()` prints the library name, city, and section
- Abstract method `displayMemberType()`

Interface: `FeeInfo`

- Method `printMembershipFee()`

Subclass: `Member`

- Inherits from `Library`
- Implements the `FeeInfo` interface
- Instance variables: `name`, `memberId`, `membershipFee`
- Constructor receives name, memberId, city, section, and membershipFee
- Override the method `displayMemberType()`
- Implement the method `printMembershipFee()`
- Method `printMemberInfo()` prints member name and member ID

Input

The input consists of several lines:

- First line: city
- Second line: section
- Third line: member name
- Fourth line: member ID
- Fifth line: membership fee
- Sixth line: updated library name

Output

Print the library information, member information, membership type, and membership fee.

The output should follow this format:

```
Library: <library name>
City: <city>
Section: <section>
Member Name: <name>
Member ID: <id>
Membership Type: Regular Member
Membership Fee: <fee>
```

Example

Examples

Test Case 1

```
Input:
Hanoi
Technology
Anna Lee
1024
50
City Library

Output:
Library: City Library
City: Hanoi
Section: Technology
Member Name: Anna Lee
Member ID: 1024
Membership Type: Regular Member
Membership Fee: 50.0
```

Test Case 2

```
Input:
Danang
Science
David Nguyen
3051
75
National Library

Output:
Library: National Library
City: Danang
Section: Science
Member Name: David Nguyen
Member ID: 3051
Membership Type: Regular Member
Membership Fee: 75.0
```

Problem 3: Smart Device Energy Tracker

A smart home system monitors different smart devices and calculates their energy usage.

Write a Java program named `SmartDeviceApp.java` that demonstrates abstract class, interface, inheritance, constructor overloading, polymorphism.

Class Requirements

Abstract Class: `Device`

- Instance variables:
  - `name`
  - `power` (watts)
  - `hours`
- Static variable:
  - `deviceCount`
- Constructors:
  - Default constructor (power = 0, hours = 0)
  - Constructor with parameters `(power, hours)`
- Methods:
  - `calculateEnergy()` that returns power × hours
  - `getDeviceCount()`
  - Abstract method `printInfo()`

Interface: `Switchable`

- Method `turnOn()`
- Method `turnOff()`

Class: `SmartFan`

- Extends `Device`
- Implements `Switchable`
- Constructor receives power and hours
- Overrides `printInfo()`

Class: `SmartLight`

- Extends `Device`

- Implements `Switchable`
- Constructor receives power and hours
- Overrides `printInfo()`

Input

The input consists of several lines:

- The first line contains an integer f presenting the number of fans.
- The next f lines each contain two numbers separated by a space:
  - the power (in watts)
  - the operating hours
- The next line contains an integer l presenting the number of lights.
- The next l lines each contain two numbers separated by a space:
  - the power (in watts)
  - the operating hours

Output

For each device created, print two lines of information in the following format:

- First line: the device name and its power in watts, format as:
  `<Device Name> running at <power> watts`
- Second line: the total energy usage calculated as `power` $\times$ `hours`, format as:
  `Energy usage:  <energy>`

After printing information for all devices, print the total number of devices instantiated in the program, format as:
`<total> devices created`

Example

Examples

Test Case 1

```
Input:
2
60 2
80 1
1
20 3

Output:
Smart Fan running at 60.0 watts
Energy usage: 120.0
Smart Fan running at 80.0 watts
Energy usage: 80.0
Smart Light running at 20.0 watts
Energy usage: 60.0
3 devices created
```

Test Case 2

```
Input:
1
50 4
2
15 2
18 3

Output:
Smart Fan running at 50.0 watts
Energy usage: 200.0
Smart Light running at 15.0 watts
Energy usage: 30.0
Smart Light running at 18.0 watts
Energy usage: 54.0
3 devices created
```